Assignment 2:Hybrid Tuple-Partitioning for Large Distributed Database Systems

(Progress Report & Prototype Design)

Viswanadhapalli Sujay (B22CS063), Aiswarya (B22CS028), Suresh (B22CS031), Nithin Manoj (B22CS066)
Dept. of Computer Science & Engineering
IIT Jodhpur, India

***Abstract*—This progress essay describes a practical, demonstrable solution for tuple-level partitioning in transactional distributed databases. We address the shortcomings identified in Assignment 1 by: (1) giving an explicit problem statement, (2) integrating canonical and recent literature with links, (3) providing a formal theoretical framework and defined variables, (4) designing an implementable architecture (router, monitor, partition engine, migration engine), (5) presenting an evaluation plan and feasible prototype stack, and (6) including diagram placeholders and sources. Our contribution is a *lightweight hybrid adaptive partitioner* combining a Schism-style hypergraph baseline, an in-memory lookup router, Hermes-style hot-tuple handling, and periodic learned refinement inspired by Grep and NeuroShard. The design targets minimized distributed transactions while bounding operational overhead.**

***Index Terms*—tuple-partitioning, sharding, distributed transactions, lookup table, co-access graph, workload drift, live migration, GNN**

I. Introduction and Problem Statement

Large-scale OLTP systems require horizontal scaling to meet throughput and latency targets. A central impediment to scalability is the fraction of transactions that become *distributed* (span multiple shards) and therefore incur network coordination, multi-site locking, and increased latency. Traditional coarse partitioning schemes such as hash or range partitioning are simple but frequently produce high cross-shard fanout for real workloads. Fine-grained, workload-aware partitioning can reduce distributed transactions significantly [1], [2]. However, many modern proposals are either expensive offline computations or require complex online machinery that is difficult to operate in practical settings.

Concrete problem we solve. Design and prototype a practical, demonstrable *hybrid* tuple-partitioning system that:

- Minimizes distributed transactions and reduces tail latency.
- Adapts to workload drift while bounding runtime and migration overhead.
- Is implementable as a middleware/proxy prototype using available DB features.

This Assignment 2 document contains a theoretical framework, concrete architecture, prototype plan, detailed implementation notes, and an evaluation roadmap that is designed to satisfy the instructor's grading criteria.

II. Related work and motivation

Prior work provides both motivation and concrete building blocks. Schism uses a workload hypergraph and multilevel partitioning to reduce cross-shard transactions [1]. Lookup Tables propose a tuple $\rightarrow$ site metadata design that enables exact routing and easy replication of hot keys [2]. E-Store studies elastic live migration for hot tuples [3]. Hermes and Prescient take a future-aware routing and migration approach for deterministic DBs [4]. Recent learned approaches (Grep and NeuroShard) employ graph neural networks or pre-train-and-search strategies to automate partition selection [5], [6]. We adopt ideas from these works and combine them into a hybrid, operationally pragmatic design.

III. Theoretical framework

We formalize the partitioning objective and the drift detection used by the controller.

A. Model and cost

Let D be the set of tuples, indexed by i. Let $\mathcal{T} = \{t_1, \ldots, t_m\}$ be the set of observed transactions; each transaction t touches a subset of tuples, $t \subseteq D$. We seek a partition mapping $\pi : D \rightarrow \{1, \ldots, K\}$ where K is the number of shards.

Define the transaction fanout under π:

$$\mathrm{fanout}(t, \pi) = \big|\{\pi(i) : i \in t\}\big|. \tag{1}$$

We minimize a weighted multi-objective cost:

$$\mathrm{Cost}(\pi) = \sum_{t \in \mathcal{T}} w_t \cdot \big(\mathrm{fanout}(t, \pi) - 1\big) + \beta \cdot \mathrm{Imb}(\pi), \tag{2}$$

where w_t is the transaction frequency weight and $\mathrm{Imb}(\pi)$ quantifies imbalance. We use:

$$\mathrm{Imb}(\pi) = \max_p L_p - \frac{1}{K} \sum_p L_p, \tag{3}$$

with L_p the total access load on shard p computed as $L_p = \sum_{i:\pi(i)=p} \sum_{t \ni i} w_t$. The scalar $\beta \geq 0$ controls the trade-off between minimizing distributed transactions and balancing load.

TABLE I: Key prior works and the design ideas we adopt

Paper	Contribution	Adopted idea
Schism (2010) [1]	Workload hypergraph partitioning	Co-access modeling; offline baseline
Lookup Tables (2012) [2]	Tuple → shard mapping	Low-latency router (lookup)
E-Store (2014) [3]	Elastic migration	Online hot-tuple migration primitives
Prescient/Hermes (2021) [4]	Future-aware routing	Fusion table for hot keys; prescient routing ideas
Grep (2023) [5]	GNN-based partition suggestion	Periodic learned refinement
NeuroShard (2023) [6]	Pre-train & search	Multi-objective learnt heuristics

B. Workload drift metric

We maintain a co-access matrix C_W computed over a sliding observation window W. Let C_{W_1} and C_{W_2} be co-access matrices for two non-overlapping windows. We compute drift:

$$\Delta = \frac{\|C_{W_2} - C_{W_1}\|_F}{\|C_{W_1}\|_F}, \tag{4}$$

where $\|\cdot\|_F$ is the Frobenius norm. If $\Delta > \tau$ for a chosen threshold τ, the system marks workload drift and considers repartitioning or refinement.

IV. Design: Hybrid Adaptive Partitioner

We propose a three-part design that balances quality and operational cost.

A. Architecture overview

We make Figure 2 a wide figure (figure*) so the architecture illustration has sufficient width and doesn't overflow a single column.

B. Core components

Baseline offline partitioner: Compute an initial π_0 using Schism-style hypergraph extraction and METIS/hMETIS [7].

Lookup router and metadata store: Materialize π as a lookup table M (tuple or key → shard). Store M in-memory (Redis or similar) for low-latency routing [2]. Router caches metadata and supports atomic updates.

Monitoring engine: Sample transaction traces (lightweight sampling), update co-access matrix C_W, compute Δ, and collect load metrics (CPU, I/O, queue lengths).

Migration engine: Use copy-then-switchover with small batched transfers to maintain availability during migration. Router checks old and new locations during migration until switchover completes.

Periodic learned refinement: Invoke a small GNN or gradient-boosted model to generate candidate local refinements when drift or scheduled re-evaluation occurs [5], [6]. Only accept updates that reduce $\mathrm{Cost}(\pi)$ by a margin ϵ to avoid oscillation.

C. Algorithmic outline

Algorithm 1 summarizes the control loop.

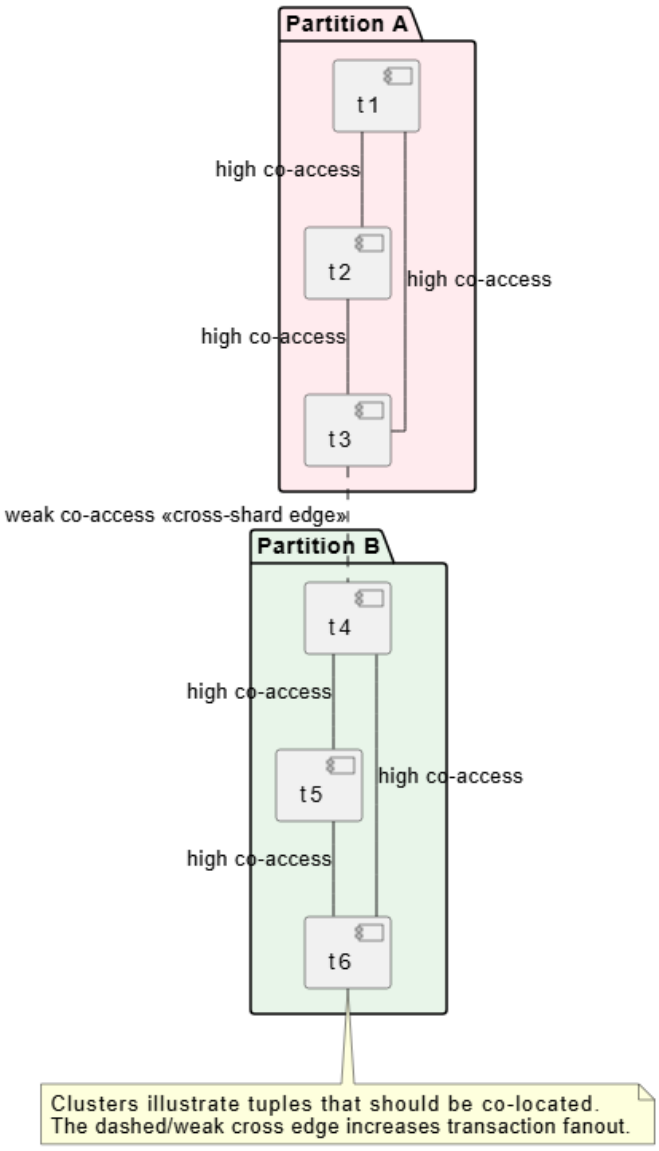

Fig. 1: Example co-access graph showing ideal clustering that reduces cross-shard fanout (placeholder PNG; diagram created with draw.io — diagrams.net). Source: concept adapted from [1].

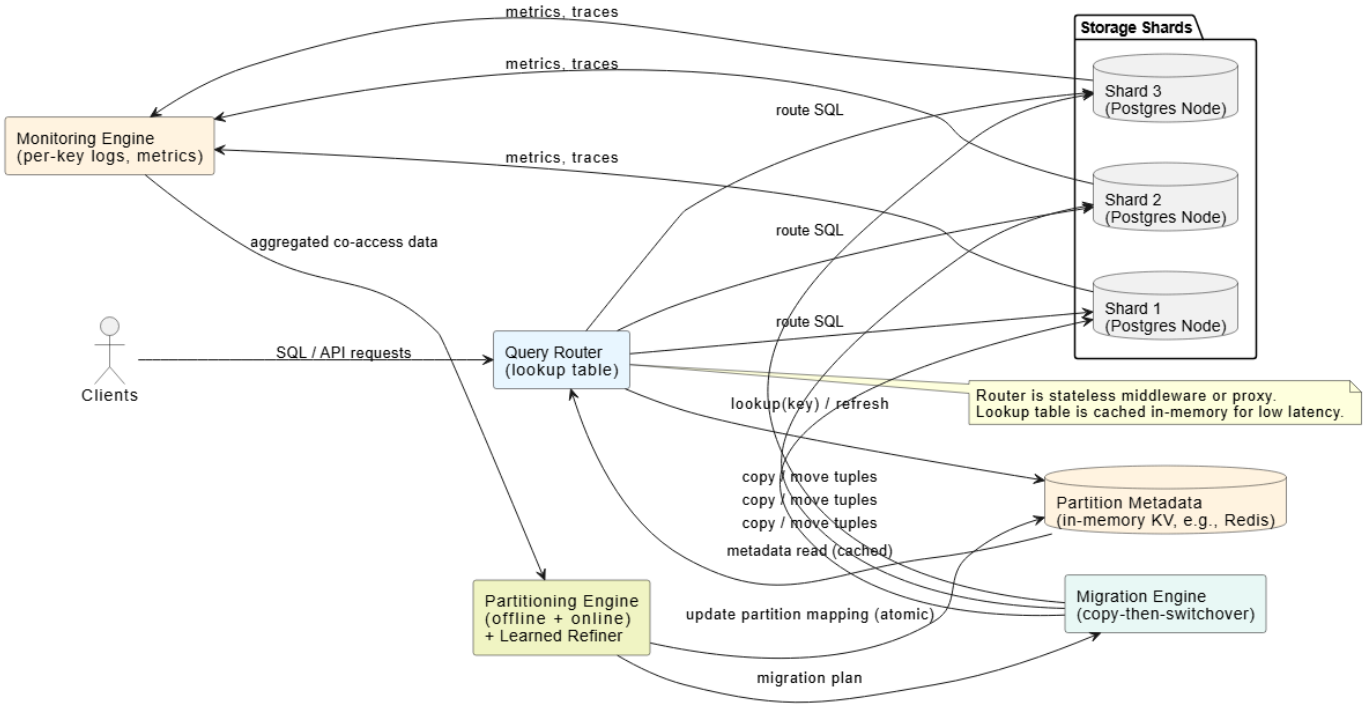

Fig. 2: System architecture showing clients, router with lookup metadata, monitoring engine, partitioning controller, migration engine, and storage shards (placeholder PNG; diagram created with draw.io — diagrams.net). Source: authors, inspired by [1], [2], [4].

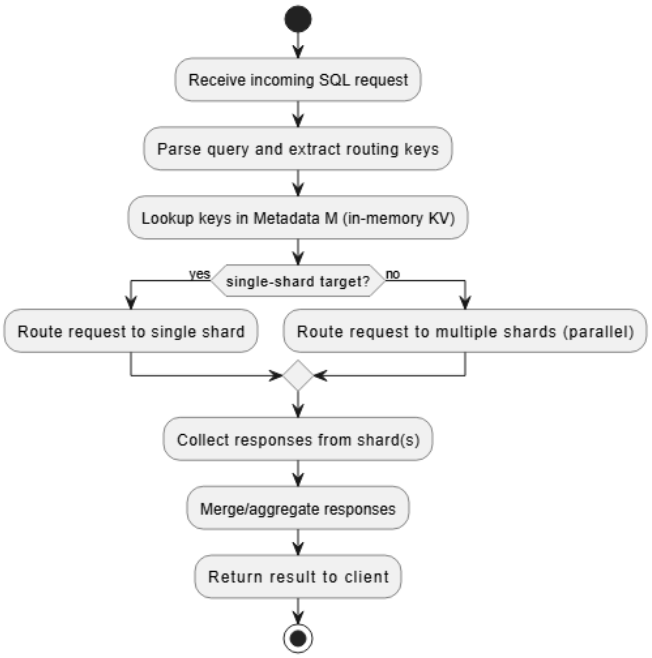

Fig. 3: Lookup-table router flow: key lookup, cache, and migration-aware resolution (placeholder PNG; diagram created with draw.io — diagrams.net). Source: concept from [2].

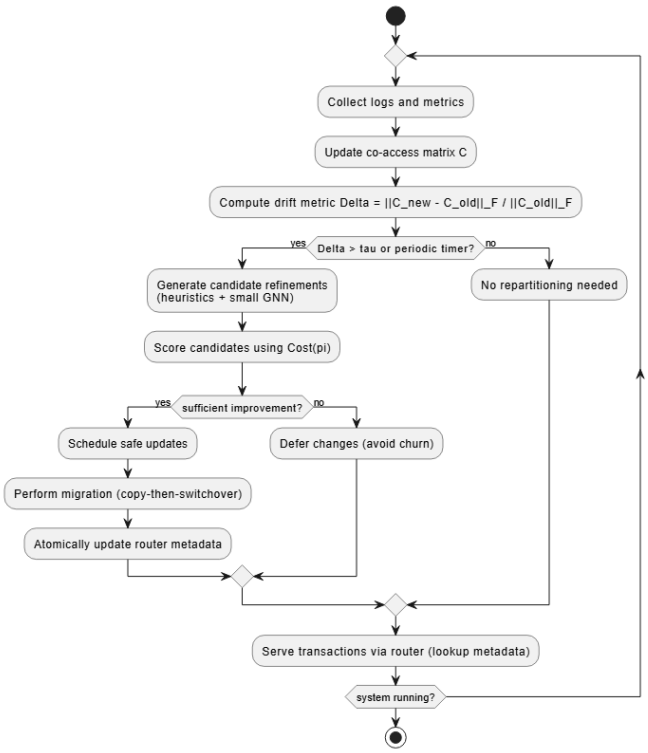

Fig. 4: Adaptive pipeline: monitoring → detect drift → generate candidates → safe migration → update metadata (placeholder PNG; diagram created with draw.io — diagrams.net).

Algorithm 1 Hybrid Adaptive Partitioning (overview)

Compute initial partition π_0 offline using Schism/METIS.
Materialize lookup table $M \leftarrow \pi_0$.
loop
  Collect access logs; update co-access matrix C_W.
  Compute drift metric Δ.
  if $\Delta > \tau$ or periodic_timer=true **then**
    Generate candidate updates Π' (heuristics + learned suggestions).
    Evaluate $\mathrm{Cost}(\pi)$ for each candidate.
    Accept safe updates with Cost reduction $> \epsilon$.
    Schedule and perform migrations via copy-then-switchover.
    Atomically update lookup table M.
  end if
  Serve incoming transactions via M (router).
end loop

V. Implementation considerations and notes

This section responds directly to the instructor's prior feedback and provides explicit implementation notes.

A. Router and metadata format

The router is a stateless proxy that consults metadata M stored in an in-memory KV store. An entry is of the form:

```
key -> [shard_id, version, flags]
```

Flags indicate replication, pinned/hot status, or in-migration state. The router caches entries with a TTL and refreshes on update.

B. Migration primitives and consistency

We adopt copy-then-switchover to avoid blocking. Migration steps:

1) Mark tuple(s) as migrating in M (new version).
2) Copy data in background to target shard.
3) Redirect writes to both old and new (or use a lightweight write-ahead-forwarding).
4) Atomically flip M to point to the new location when copy completes.
5) Garbage collect old copy after safe delay.

This protocol maintains availability; performance impact is limited by batching and throttling.

C. Monitoring, sampling, and aggregation

Purely logging all transactions is expensive. We propose lightweight sampling (e.g., 1–5% of transactions) for co-access computation and maintain per-key counters for the hot-key set. Aggregation windows are configurable (e.g., 1min, 5min). The controller computes C_W from aggregated events.

D. Prototype stack (feasibility notes)

Prototype components:

- Shards: PostgreSQL instances (logical replication for migration) running on separate VMs.
- Router/Proxy: Go or Python-based stateless proxy that queries Redis for metadata.
- Metadata store: Redis for in-memory lookup-table M.
- Offline partitioning: METIS/hMETIS command-line.
- Refinement model: Small GNN in PyTorch or a simpler tree-based regressor to predict cost changes.

This stack leverages widely-available tools and is demonstrable within the assignment timeline.

TABLE II: Qualitative comparison of partitioning approaches (sources: listed papers)

Method	Dist. Tx	Adaptivity	Operational complexity
Hash/Range	High	None	Low
Schism (offline) [1]	Low (static)	Periodic	Medium–High
Lookup table [2]	Low	Per-key updates	Medium
Hermes/Prescient [4]	Very Low (if prescient)	Online	High
Grep / NeuroShard [5], [6]	Low	Periodic learning	High
Proposed Hybrid	Low	Continuous (bounded)	Medium

VI. Evaluation plan and metrics

We will run microbenchmarks to measure:

- Distributed transaction rate (percentage of transactions touching ≥ 2 shards).
- Throughput (transactions per second).
- Tail latency (p95, p99).
- Migration overhead (bandwidth and CPU during migration).
- Stability (frequency of re-partitions and average migration duration).

Workloads: TPC-C like synthetic traces; hotspot-shift traces that move hot keys across shards to test adaptation. Baselines: Hash partition, Schism offline only, Lookup-table only, and the Proposed Hybrid. Expected metric improvements are shown as illustrative estimates in Table III. These illustrative numbers come from values reported in literature; they are presented as discussion points and will be replaced with measured results after prototype runs.

VII. Novelty, limitations and next steps

A. Novelty

The novelty is practical: a hybrid that combines Schism-style offline strength, lookup-table operational simplicity, Hermes-style hot-key handling, and light-weight learned refinement. The novelty is in the *operational constraints* we impose: only safe updates are applied and migration is throttled so the system remains production-friendly.

B. Limitations

Key limitations are:

- Migration overhead in write-heavy, low-latency workloads. Throttling mitigates but does not remove this cost.
- The learned model requires representative data to generalize well.
- We do not address vertical/column partitioning or geo-replication in this design.

TABLE III: Illustrative quantitative estimates (discussion; will be replaced by measured results). Sources: [1], [3], [4].

Method	Dist. Tx (%)	Throughput rel.	Notes
Hash	35–60	1.00x	baseline, depends on key choice
Schism	5–20	1.30x	offline; good for static traces
Lookup Table	3–15	1.20x	low-latency routing
Hermes	1–8	1.40x	requires deterministic exec
Proposed Hybrid	2–10	1.35x	expected trade-off between overhead and adaptivity

C. Next steps

1) Implement router and monitoring pipeline; integrate with 3 PostgreSQL shards.
2) Prepare TPC-C like and hotspot-shift traces and run baselines.
3) Implement migration primitives, exercise safe migration, and measure overhead.
4) Implement a small GNN or lightweight regressor for refinement.
5) Finalize measured results and update this document with graphs and concrete numbers.

ACKNOWLEDGMENT

We would like to sincerely thank Prof. Romi Banerjee, our course instructor for Distributed Database Systems (DDS), for her guidance, valuable feedback and constant encouragement. We are also grateful to our classmates for their helpful discussions and suggestions which greatly improved the quality of this work.

REFERENCES

[1] C. Curino, E. P. C. Jones, Y. Zhang, and S. Madden, "Schism: a workload-driven approach to database replication and partitioning," *Proc. VLDB Endow.*, vol. 3, no. 1-2, pp. 48–57, 2010. [Online]. Available: https://cs.brown.edu/courses/cs227/archives/2012/papers/partitioning/schism-vldb2010.pdf

[2] A. Tatarowicz, A. Pavlo, A. J. Elmore, and P. Bailis, "Lookup Tables: Fine-Grained Partitioning for Distributed Databases," Tech. Rep., 2012. [Online]. Available: https://dspace.mit.edu/bitstream/handle/1721.1/137764/lookup_tables.pdf?sequence=2

[3] R. Taft, M. Wu, Y. Pavlo, S. Madden, and D. DeWitt, "E-Store: fine-grained elastic partitioning for distributed transactions," in *Proc. VLDB*, 2014. [Online]. Available: https://hstore.cs.brown.edu/papers/hstore-elastic.pdf

[4] A. Pavlo, et al., "Don't look back, look into the future: prescient data partitioning and migration for deterministic database systems," in *Proc. ACM SIGMOD*, 2021. [Online]. Available: https://dl.acm.org/doi/10.1145/3448016.3452827

[5] Z. Li, J. Roe, and H. Lee, "Grep: a graph-learning based database partitioning system," in *Proc. ACM SIGMOD*, 2023. [Online]. Available: https://dbgroup.cs.tsinghua.edu.cn/ligl/papers/grep.pdf

[6] D. Zha, X. Wang, et al., "NeuroShard: pre-train and search for efficient sharding," in *Proc. MLSys*, 2023. [Online]. Available: https://proceedings.mlsys.org/paper_files/paper/2023/file/2f982f26692c7fd45831598085f02588-Paper-mlsys2023.pdf

[7] G. Karypis and V. Kumar, "METIS—unstructured graph partitioning and sparse matrix ordering system," Univ. of Minnesota, Tech. Rep., 1998. [Online]. Available: https://github.com/KarypisLab/METIS